

Parallel Betweenness Centrality via Brandes' Algorithm

Analysis of a multi-core CPU, OpenMP-based approaches on different graph topologies and machine architectures

Jerry Cheng and Julie Wu

Background

Given graph $G = (V, E)$ find for all vertices...

Betweenness centrality: The frequency that a node is on the shortest path between pairs of other all other nodes.

$$BC(v) = \sum_{s \neq v \neq t \in V} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

$\sigma_{st}(v)$ = # of shortest paths between (s, t) passing through $v \in V$.

σ_{st} = # of shortest paths between $(s, t) \in V \times V$



Brandes' Algorithm

```
for each  $s$  in  $V$ :  
    forward pass:  
        run BFS from source  $s$ , finding  $s$ - $t$  shortest paths  
        track vertices in each frontier layer  
  
    backward pass:  
        from farthest to closest layer to  $s$ , for each node:  
            update all predecessor nodes' BC scores using own
```

Inputs / Test Data

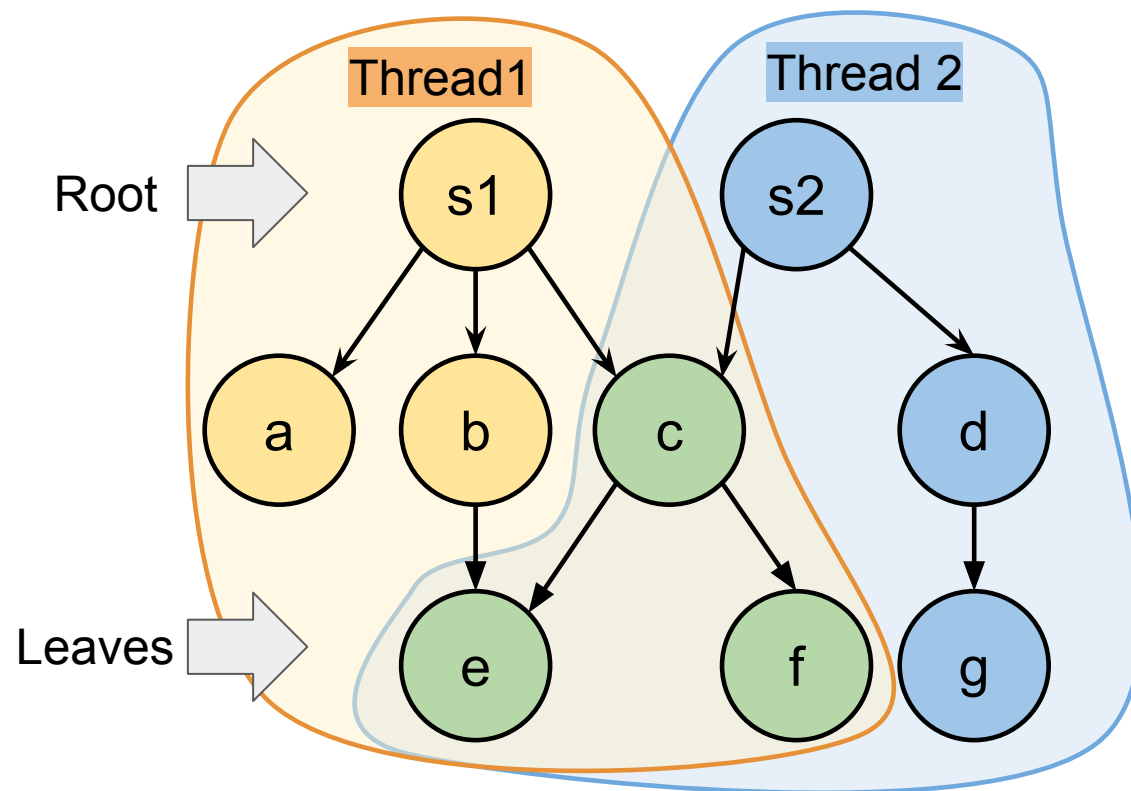
Two contrasting graph types of S, M, and L sizes:

- High-diameter graphs (e.g. road networks)
Result in very deep BFS trees with small frontiers.
- Scale-free graphs (e.g. social networks)
Power-law scaling with a few very influential nodes. Result in short BFS trees with large frontiers.

Handled as 1-D arrays using compressed sparse row (CSR) format

Method: Across Brandes' (Coarse Approach)

- Parallelize over multiple, independent single-source Brandes' algorithm iterations for different source vertices
- Synchronization:
 - Atomic updates to partial BC metric per node
- Workload imbalance from differently sized BFS traversals



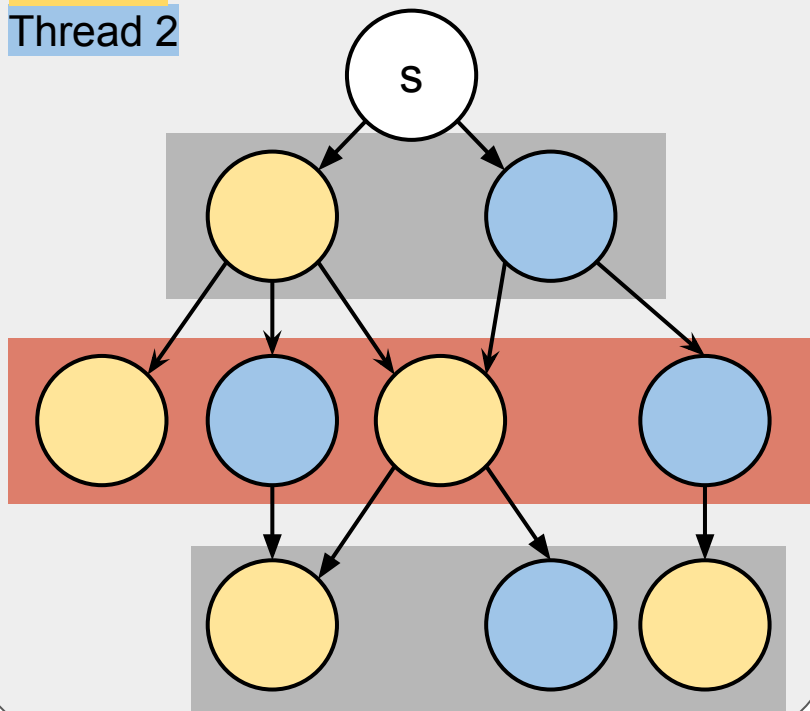
Independent runs of Brandes'* on sources s1 and s2 with two threads. BFS is embarrassingly parallel. On backward pass, BC updates to c, e, and f are under contention.

Method: Within Brandes' (Fine Approach)

- Parallelize within a single-source Brandes' algorithm iteration
- Synchronization: strict barrier required at every depth level
- Scheduling: workload imbalance from “hub” nodes in the frontier

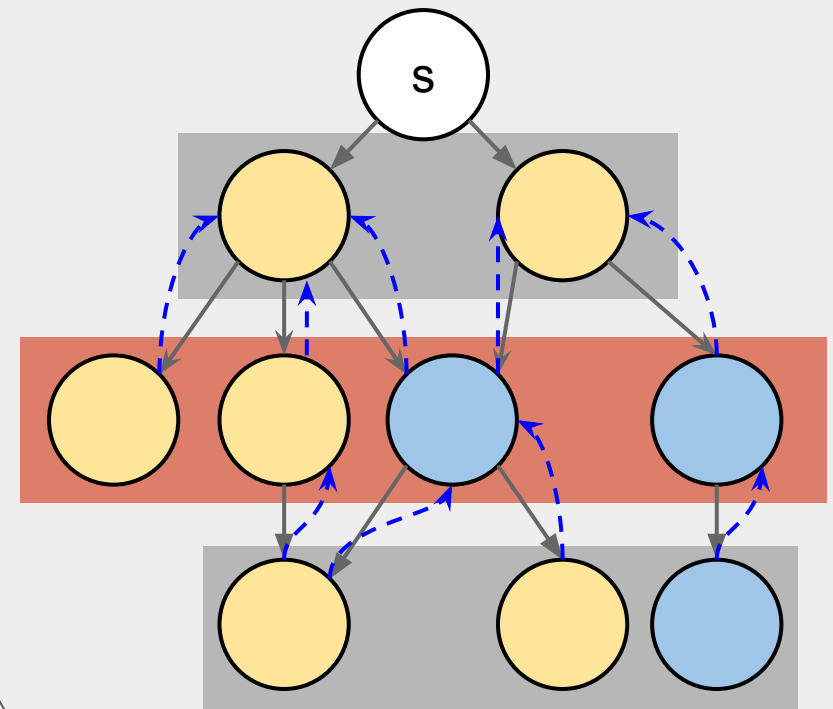
Forward pass: BFS

Thread1
Thread 2



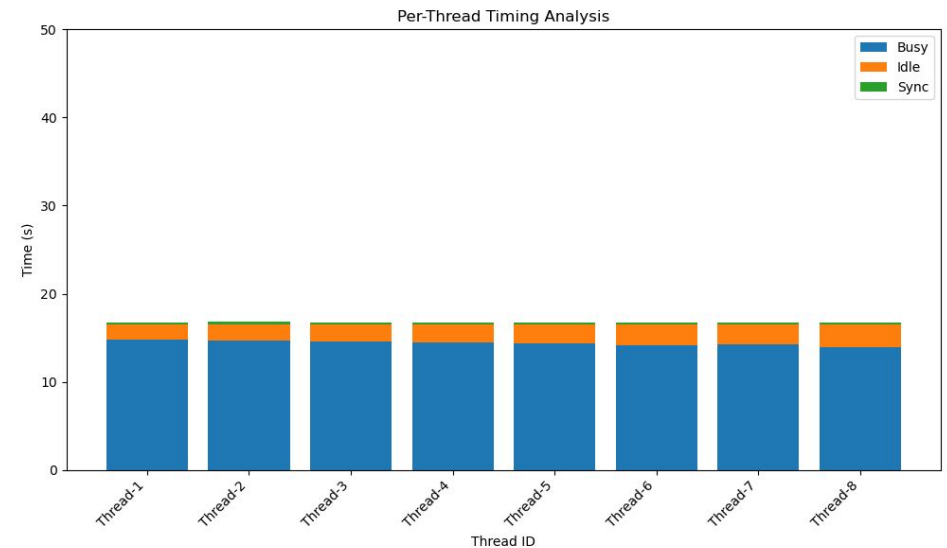
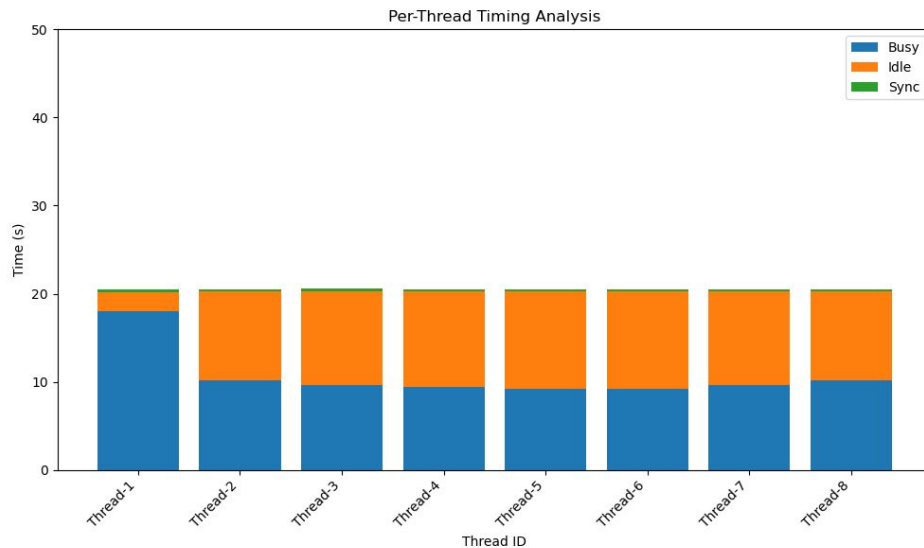
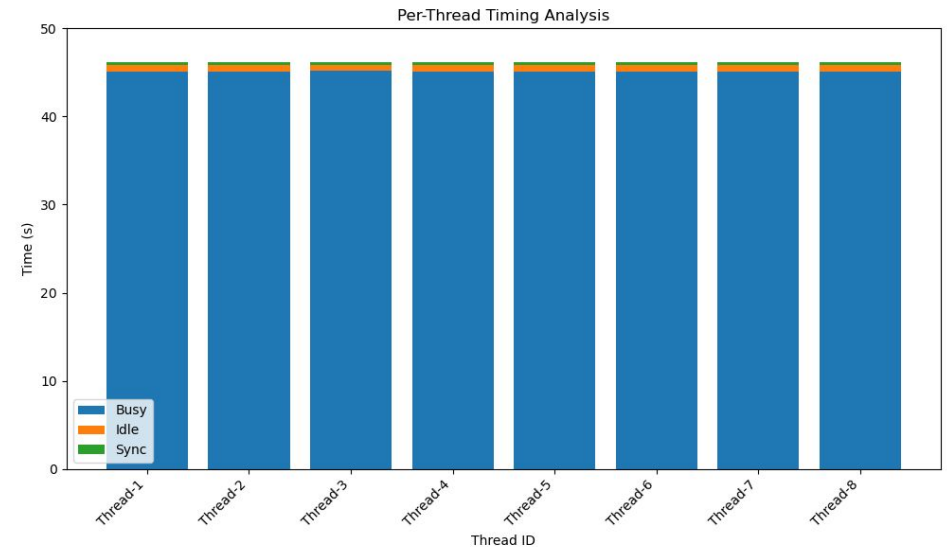
Source
↓ ↑
Layer 1
↓ ↑
Layer 2
↓ ↑
Layer 3

Backward pass



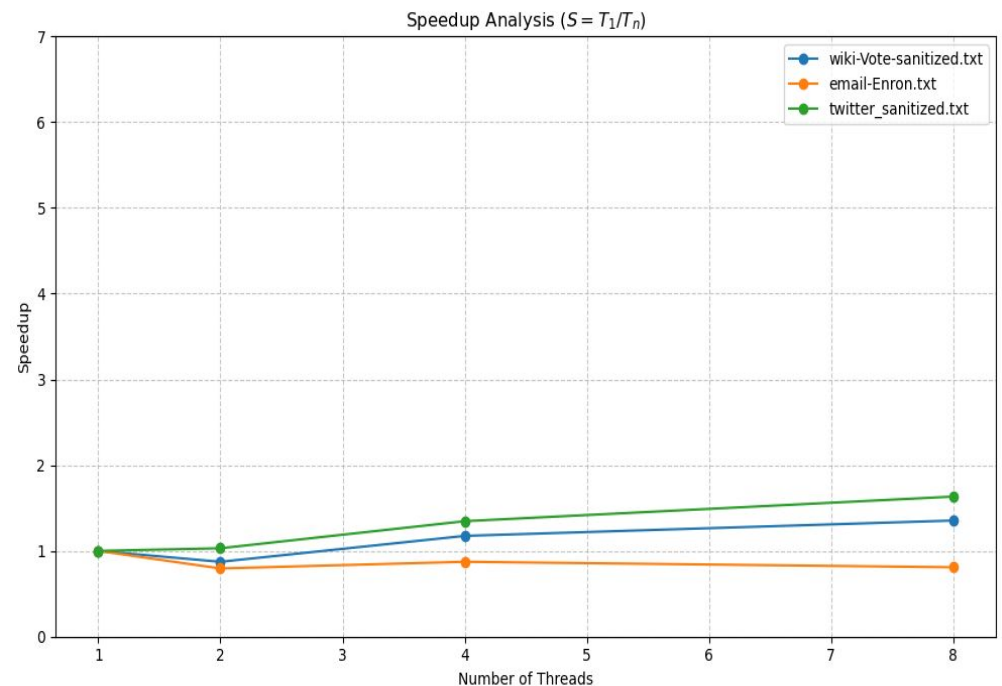
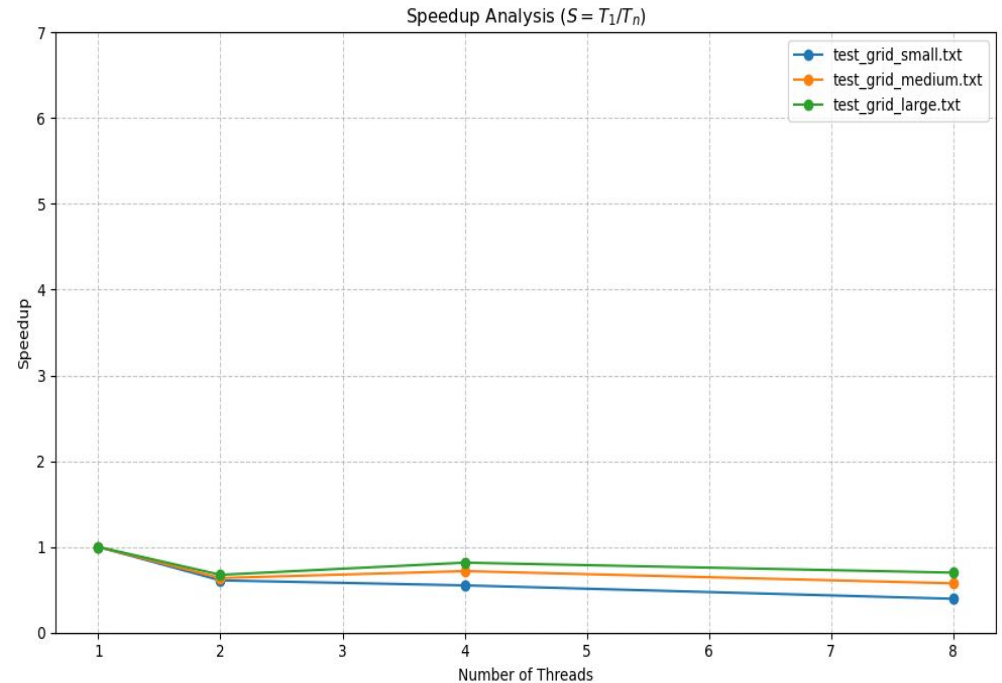
Load Balance Testing

- Static-blocked
 - Poor load balance
- Dynamic
 - Too much overhead
- Static-interleaved
 - Distributes dense “hub” nodes



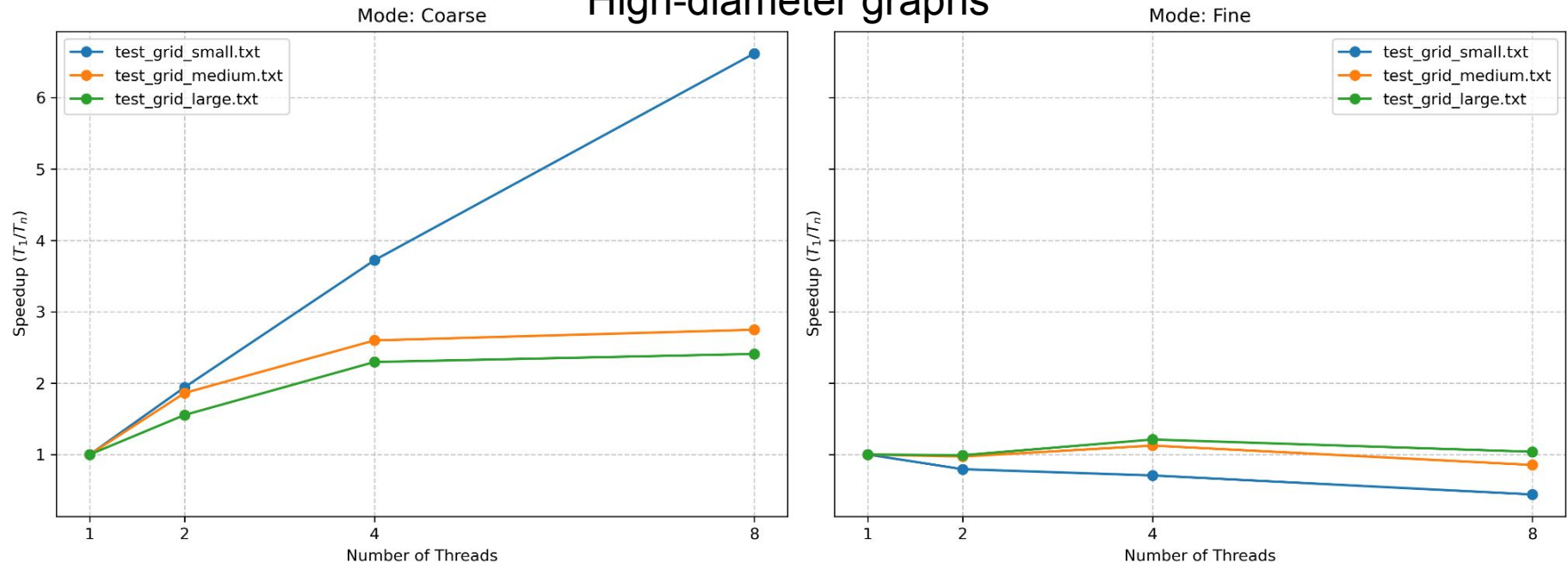
Naive Speedup

- Naive level-synchronous implementation showed slowdown on most testcases
- Optimized level-synchronous code shows better performance on larger graphs
- Source-parallel implementation sees better performance on PSC machines since they have significantly more memory

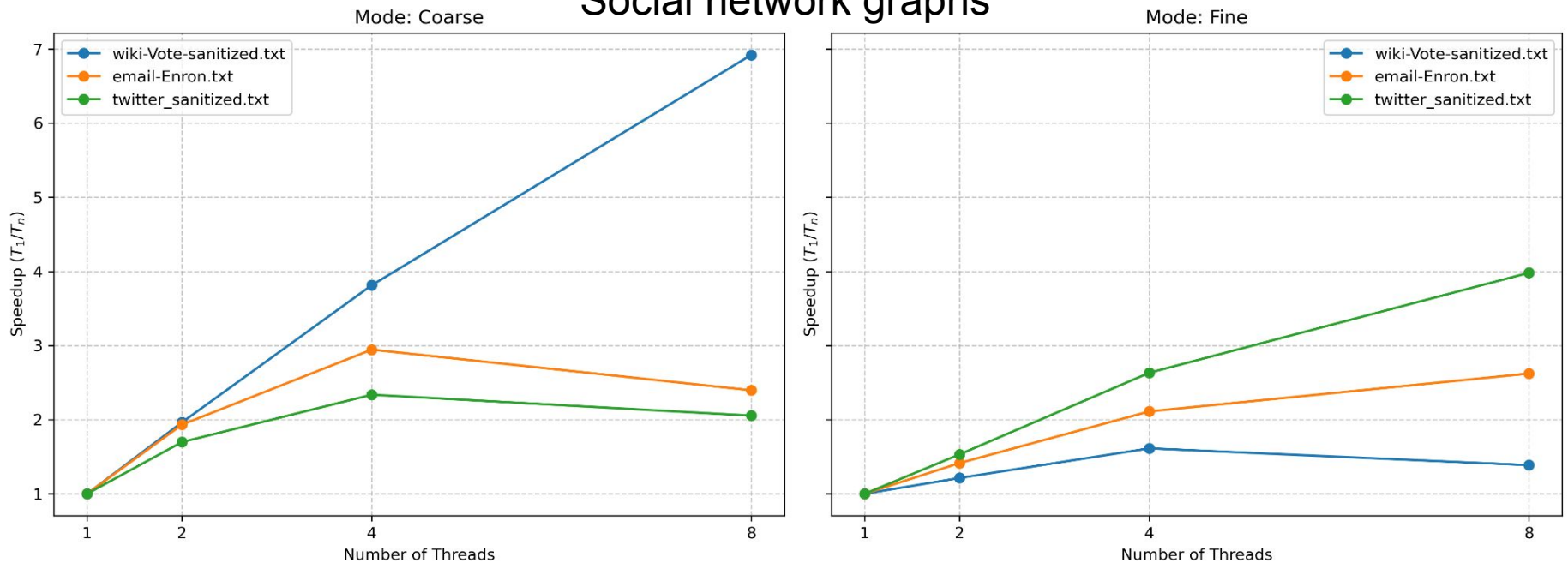


Speedup Results: GHC Machines

High-diameter graphs

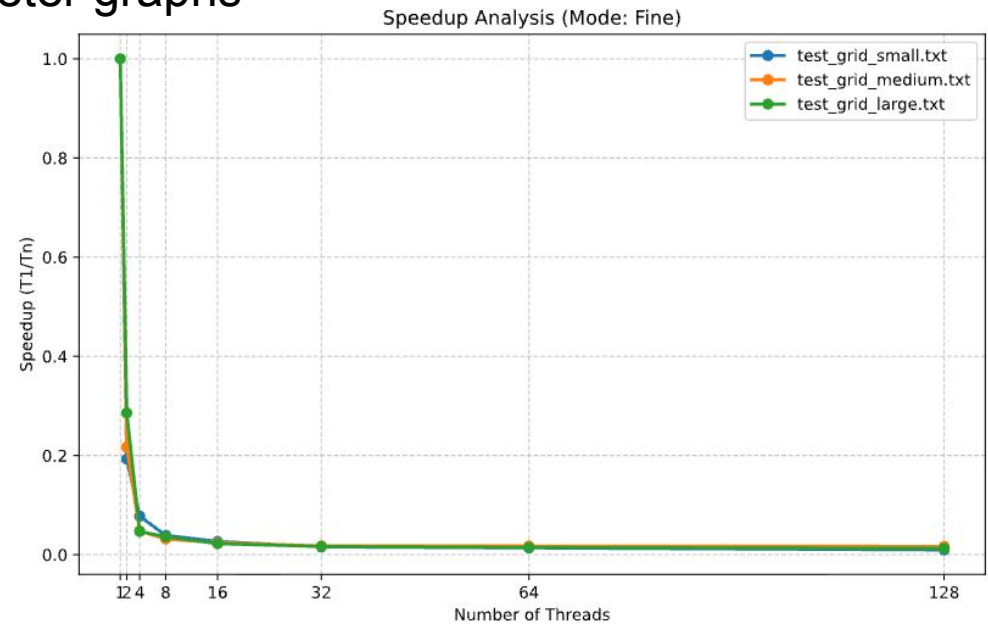
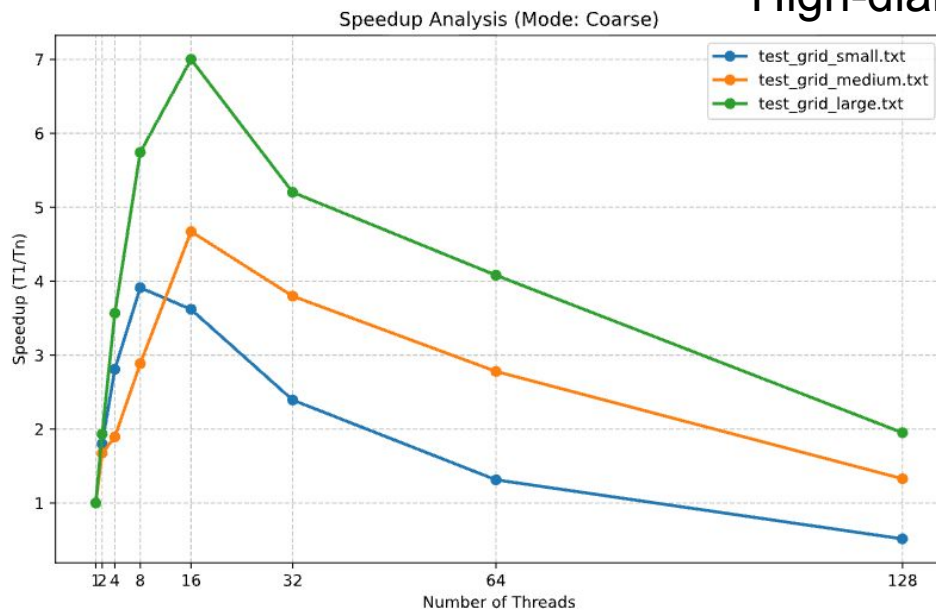


Social network graphs



Results: PSC RM Nodes → NUMA Architecture

High-diameter graphs



Social network graphs

